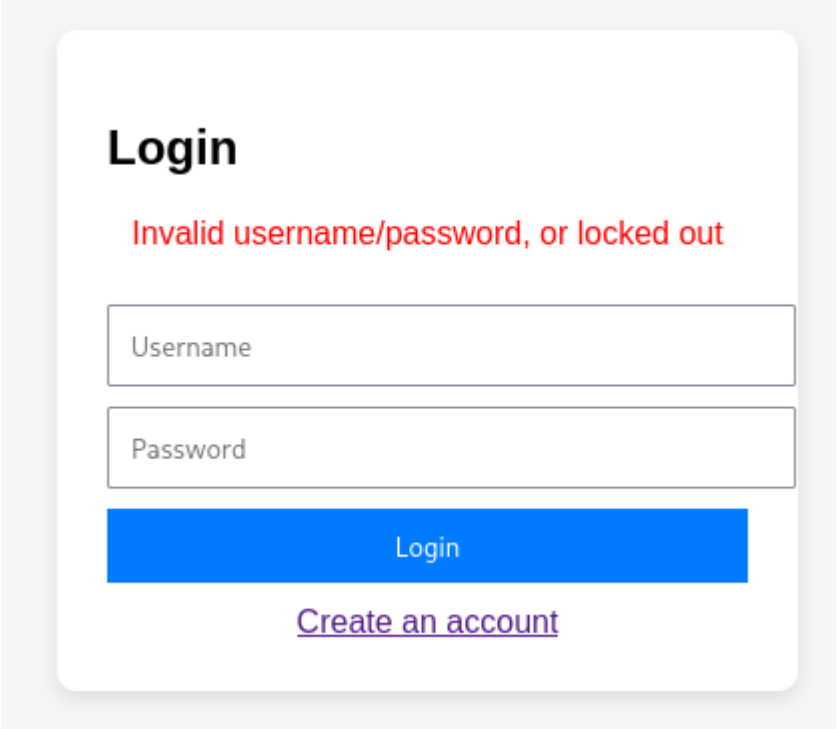
Penetration Test Report

Authentication Testing

Bruteforce Protection

When attempting to bruteforce a user's password, the account is locked after 5 attempts. Along with this, the response from the server does not specify whether our attempted password was accepted and marked as incorrect, or if our login attempt was ignored from the account being locked.

With our knowledge of the site locking an account after 5 attempts, this may not mean much. However, to an attacker who may not know the lockout policy, this can prove very effective in confusing them on whether the account was *truly* locked out, or if the password was just not accepted.



However, with this in mind, there *is* a timing attack vulnerability. In this example, I specify an obviously incorrect password for both the username "admin" (a real user), and the username "abcdefg" (a nonexistent user). Both give the failure message, but the "admin" message takes much, *much* longer to complete.

An attacker can use this, with the nonexistent rate limit policy, to enumerate usernames on the site.

```python
from requests import post
from time import time

url = "https://localhost:5000/login"

def log_time(uname):
    t = time()
    post(url,
        data={"username":  uname, "password": "obviouslyincorrect"},
    verify=False)
    return time() - t

print(log_time("abcdefg"))
print(log_time("admin"))
```

```
┌──(root㉿kali)-[~/compsec_project]
└─# python3 test.py
/usr/lib/python3/dist-packages/urllib3/conn
/en/latest/advanced-usage.html#tls-warnings
  warnings.warn(
0.043905019760131836
/usr/lib/python3/dist-packages/urllib3/conn
/en/latest/advanced-usage.html#tls-warnings
  warnings.warn(
0.24117827415466309
```

Password Complexity Enforcement

tests/test_registration.py tests for password complexity, and the current password complexity policy is 1 special, 1 lowercase, 1 uppercase, 1 digit, and atleast 12 characters.

This is better than no password policy, but not the best. Ideally, a password policy should be much harsher.

Session Management

```python
response.set_cookie(
    'session_token',
    token,
    httponly=True, # Prevent JavaScript access
    secure=True, # HTTPS only
    samesite='Strict', # CSRF protection
    max_age=1800 # 30 minutes
)
```

```python
def validate_session(self, token):
    """Check if session is valid"""
    sessions = self.load_sessions()
    if token not in sessions:
        return None

    session = sessions[token]

    # Check timeout
    if time.time() - session['last_activity'] > self.timeout:
        self.destroy_session(token)
        return None

    # is the user inaccessible?
    if login_manager.get_user(session["user_id"]) == None:
        self.destroy_session(token)
        return None

    session['last_activity'] = time.time()
    sessions[token] = session
    self.save_sessions(sessions)
    return session
```

```python
def __init__(self, timeout=1800): # 30 minutes
    self.timeout = timeout
    self.storage = EncryptedStorage("data/sessions.json")
```

As can be seen here, the session of a user is set to 30 minutes maximum, after which a session is no longer marked as valid. However, in the case of a user lockout, while the sessions *are* discarded in the midst of a lockout; they are not eliminated preemptively.

While not a huge concern, if a user has a session token lasting longer than 15 minutes, they can simply wait out the lockout, then continue as normal once it completes without their session being revoked.

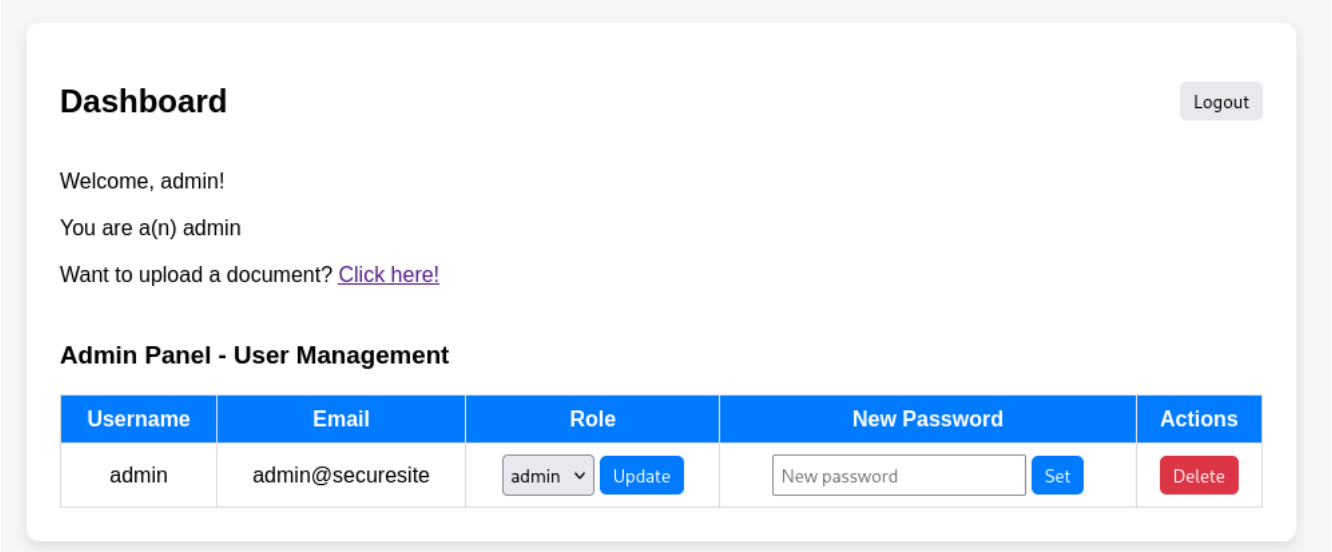
In addition, there exists no ability for a logged in user to revoke all other sessions in their account's name. This means, if someone thinks their account is being compromised, they can essentially do nothing about it. This *does* mean an attacker cannot revoke the actual user's session on the contrary, but it is something worth considering.

Logout Functionality

```python
@app.route("/logout", methods=["POST"])
@require_auth
def logout():
    # delete cookie and destroy session
    resp = make_response(redirect("/login"))

    # destroy the session token
    token = request.cookies.get("session_token")
    session_manager.destroy_session(token)

    # mark the token as no longer valid
    resp.delete_cookie("session_token")
    return resp
```



There does exist logout functionality. Upon logout, the session itself is destroyed, preventing it from being used again.

Password Reset Security

For a non-administrative user, there does not exist the ability for someone to change their own password. This is very dangerous incase the user is currently using a password they may have reused from an unrelated data breach, as an example.

Administrative users *can* reset passwords of regular users, as shown above. The administrative user can change a user's password to anything, including something which would otherwise not been allowed as per the password policy.

This is potentially very dangerous.

Authorization Testing

Horizontal/Vertical Privilege Escalation

In the event of a user, say for example the admin user, having their password known by an attacker, it is able to be easily compromised. Because of no multifactor authentication being built into the site, there is no extra barrier of entry for such attacker.

Direct Object Reference

As of note, there is no existence of an IDOR vulnerability within the site, apart from the intentional functionality of the **share_code** for a document.

Forced Browsing

The only endpoint that can be enumerated would be the document endpoints. However, because a document ID is essentially randomly generated, this would be very hard to enumerate.

```
@app.route("/document/<doc_id>", methods=["GET"])
def get_document(doc_id):
```

```
def upload_document(self, raw_data, doc_name):
    data = self.storage.load_encrypted()

    doc_id = urandom(16).hex()
    data[doc_id] = {
        "owner": self.session_data["username"],
        "data": raw_data,
        "name": doc_name
    }

    self.storage.save_encrypted(data)
    return doc_id
```

Input Validation Testing

XSS

Because the site only sends a response via using Jinja2, which automatically sanitizes XSS by default, there are no XSS vulnerabilities.

Path Traversal

Because of the way documents are stored (in a keyed database, referenced by a document ID), there is essentially no interaction between a user and the filesystem. Thus, path traversal doesn't have any attack surface.

Command Injection

There are no calls to **os.system()**, thus meaning command injection is not possible.

File Upload Vulnerabilities

Because files uploaded by users are essentially just stored as strings (encoded in base64), there is no extra attack surface a potential attacker can use. The strings are simply stored, and then if requested, they are returned back to the user as an **octet-stream**.

Session Security

Session Fixing

Because a session token is essentially randomly generated, there is no way for an attacker to pre-set a session ID.

```
def create_session(self, user_id):
    """Create new session token"""
    token = secrets.token_urlsafe(32)
```

Session Hijacking

If a user's session were to be compromised, there exists no ability for a currently logged in user to disable all sessions and force universal reauthentication. Thus, an attacker with a valid session, due to the independent nature of all sessions, is allowed to continue exploitation without much worry.

If an admin were to reset a user's password, it does not force all sessions to re-login. The only way to fix this vulnerability is for the admin to delete a user's account.

This is easy to fix by introducing at the very least a "logout user" button on an administrator's panel. However, ideally, a user should be able to force all sessions to deauthenticate.

Concurrent Session Handling

There is, as of now, no policy on concurrent sessions. A user can have unlimited sessions active at the same time.

A policy should be put into place enforcing a policy on concurrent sessions, such as limiting them.

Session Timeout

Session timeout is set to 30 minutes at absolute maximum, a good-enough time for a session.

Configuration Testing

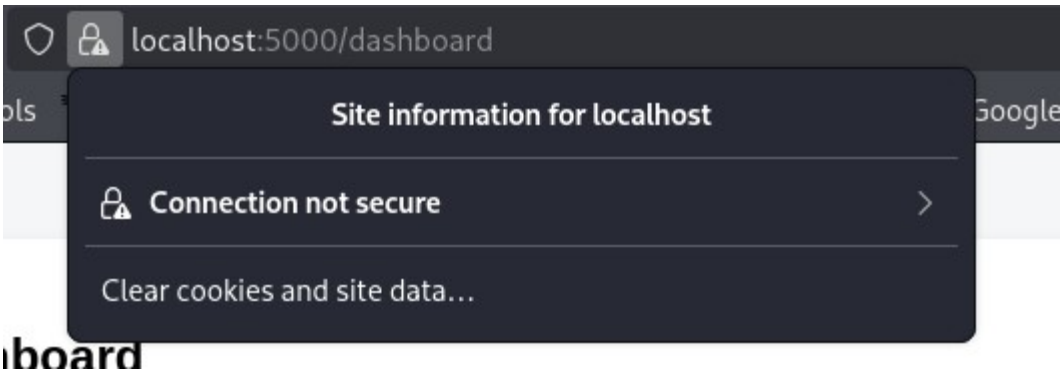
Security Headers

Per the example code on the project writeup, code making sure to add secure headers is implemented. Because of concerns with my local home network, I was not able to publically host this and perform more comprehensive header testing.

TLS/SSL

TLS/SSL is implemented, with a self-signed cert, on the python application. Ideally, this should be done with a reverse proxy such as **nginx**, with the flask server just serving HTTP that the **nginx** server, to the outside internet, handles all SSL for.

The certificate is self-signed, which is not ideal and should instead be granted from a valid certificate authority.



Error Handling

In the event of an error, flask by default returns a **500** to the user, prints the stacktrace to stdout, and resumes execution.

While an attempt has been made to significant reduce, if not eliminate, errors on the website, this functionality serves as a backup to prevent the site from terminating incase one slip-up in programming occurs.

Debug Mode

The server is run in production mode (in context of a flask single-threaded server). Thus, the debug endpoint and console are not exposed.

```
┌──(root㉿kali)-[~/compsec_project]
└─# ./run_server.sh
Initialized admin user 'admin' with
It is recommended to memorize this p
 * Serving Flask app 'app'
 * Debug mode: off
```